Kevin Küster

Zusammenfassung: RSys_Kl

Rechnersysteme

Petrinetze

Schaltregeln

▷ Aktivierungsbedingung: $\forall p \in \epsilon : M(p) \geq I^-(p,t)$

Transition aktiviert in Markierung M, wenn jede Vorstelle p mindestens so viele Marken enthält wie das Gewicht der Kante $p \to t$ vorschreibt

▷ Schaltbedingung (Feuern): $\forall p \in P : M'(p) = M(p) - I^-(p,t) + I^+(t,p)$

Beim Schalten werden $I^-(p,t)$ aus jeder Vorstelle entnommen und $I^+(p,t)$ Marken in jede Nachstelle gelegt

Erreichbarkeitsgraph

(i) Startmarkierung M_0 als Vektor (Belegung der Stellen)

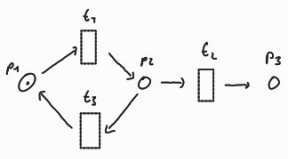
(ii) Jede Transition auf Aktivierung prüfen

(iii) Für jede aktivierte Transition M' aufstellen

↳ Kante $M \to t \to M$ im Graphen einfügen

(iv) Wiederhole bis keine neuen Markierungen

Beispiel



$M_0 : (1,0,0)$

↳ $t_1 \to M_1 = (0,1,0)$

$M_1 : (0,1,0)$

↳ $t_2 \to M_2 = (0,0,1)$

↳ $t_3 \to M_0 = (1,0,0)$

$M_2 = (0,0,1)$

↳ Deadlock

Erreichbarkeitsgraph: $M_0 \rightleftarrows M_1 \to M_2$

Beschränktheit & Sicherheit

▷ Beschränkt: Netz k-beschränkt, wenn es $k \in \mathbb{N}$ gibt, sd für jede erreichbare Markierung M und jede Stelle p gilt: $M(p) \leq k$

▷ Sicher: Netz ist 1-beschränkt

⇒ Beschränkt ⇔ endlich viele erreichbare Markierungen

Vorgehen: Für jede Stelle p Maximum der Tokenzahlen über alle erreichbaren Markierungen ablesen ⇒ maximale Obergrenze = k

Lebendig & Verklemmungsfrei

▷ Verklemmungsfreiheit: In jeder erreichbaren Markierung ist mindestens eine Transition aktiviert

▷ Lebendigkeit: Jede Transition kann von jeder erreichbaren Markierung aus (ggf. über Sprünge) wieder aktiviert werden

⇒ Lebendigkeit ⇒ Verklemmungsfreiheit

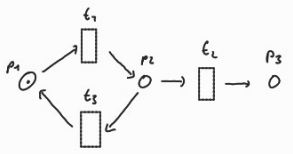
Inzidenzmatrix & P-Invarianten

▷ C^+: Vorwärts-Inzidenzmatrix: $C^+(p,t) = I^+(p,t)$

▷ C^+: Vorwärts-Inzidenzmatrix: $C^+(p,t) = I^+(p,t)$

▷ C: Gesamt-Inzidenzmatrix: $C = C^+ - C^-$

Beispiel



$C = C^+ - C^-$:

	t_1	t_2	t_3
p_1	-1	0	1
p_2	1	-1	-1
p_3	0	1	0

C^- :

	t_1	t_2	t_3
p_1	1	0	0
p_2	0	1	1
p_3	0	0	0

C^+ :

	t_1	t_2	t_3
p_1	0	0	1
p_2	1	0	0
p_3	0	1	0

▷ P-Invariante: Vektor $v \in \mathbb{Z}^n$, $v \neq 0$ ist place(P)-Invariante gdw:

$$v^T \cdot C = 0$$

⇒ Gewichtete Tokensumme $v^T \cdot M$ bleibt konstant für jede erreichbare Markierung

⇒ T-Invariante: Vektor $f \geq 0 : C \cdot f = 0$

↳ Wenn Transition entsprechend Vektor f schaltet (t_i wird f_i-mal gefeuert) → Wieder an Ausgangsmarkierung

Protokollinstanzautomaten

Signaltypen

- +x: Senden (Output) → PIA erzeugt Signal x; Partner muss gleichzeitig empfangen → x geht in Puffer
- -x: Empfangen (Senden): PIA konsumiert Signal x; Partner muss gleichzeitig senden → x wird aus puffer genommen
- Intern: Interne Aktion ohne Kommunikationspartner

Produktautomat

- **Direkte Kopplung**: Signale müssen synchron von beiden PIAs gefeuert werden; Wenn kein Partner verfügbar ist → Blockierung
  - ↳ S_1: +x, S_2: -x
- ↳ Produktzustand: Jeder Zustand im Produktautomaten ist Paar (s_1, s_2); Transitionen entstehen nur, wenn beide PIAs zusammen feuern können
- ↳ Transitionen:

(i) $S_1 \to S_2$: $\exists q_1' : (q_1, +x, q_1') \in \delta_1 \wedge \exists q_2' : (q_2, -x, q_2) \in \delta_2$
↳ $(q_1, q_2) \to [x] \to (q_1', q_2')$

(ii) $S_2 \to S_1$ Analog

(iii) S_1 Intern: $\exists q_1' : (q_1, a, q_2) \in \delta_1 \wedge a \notin K \to (q_1, q_2) \to [a] \to (q_1', q_2)$

(iv) S_2 Intern Analog

- **Kanalkopplung** (indirekte Kopplung): PIAs kommunizieren asynchron über gepufferte Kanäle; S_1 legt Signal in Kanal, S_2 holt es zu einem späteren Zeitpunkt heraus und umgekehrt
  - ↳ Kanäle arbeiten nach FIFO (First In - First Out) - Prinzip
  - ↳ Produktzustand bei Kanalkopplung:

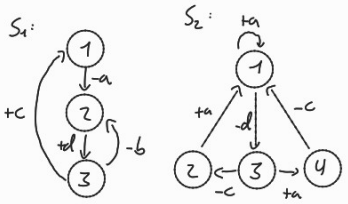
$(S_1, ch_{12}, ch_{21}, S_2)$ S_1: Zustand S_1; S_2: Zustand S_2;
ch_{12}: Inhalt Kanal $S_1 \to S_2$ (FIFO-Liste);
ch_{21}: -"- $S_2 \to S_1$

Startzustand: $(q_{10}, [\,], [\,], q_{20})$

Eigenschaften

- Deadlock - Zustand, aus dem keine Transition feuern kann
- Endlosschleife - Unendlicher Zyklus $z \to \dots \to z$
- Lebendigkeit - Jeder erreichbare Zustand hat mind. einen Nachfolger
  ⇒ Keine Deadlocks

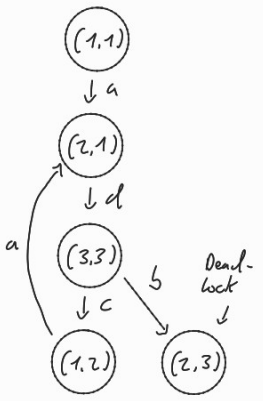
Beispiel



$(1, -a, 2)$,
$(2, +d, 3)$,
$(3, -b, 2)$,
$(3, +c, 1)$

$(1, +a, 1)$,
$(1, -d, 3)$,
$(2, +a, 1)$,
$(3, -c, 2)$,
$(3, +a, 4)$,
$(4, -c, 1)$

Direkte Kopplung - Produktautomat:



Zyklus bei Kanalkopplung:

$(1, [\,], [\,], 1) \xrightarrow{S_2:+a} (1, [\,], [a], 1) \xrightarrow{S_1:-a} (2, [\,], [\,], 1) \xrightarrow{S_1:+d} (3, [d], [\,], 1) \xrightarrow{S_2:-d} (3, [\,], [\,], 3) \xrightarrow{S_1:+c} (1, [c], [\,], 3) \xrightarrow{S:-c} (1, [\,], [\,], 2) \xrightarrow{S_2:+a} (1, [\,], [a], 1)$

Logische Zeitstempel

In verteilten Systemen keine gemeinsame Uhr - Jeder Prozess hat eigene; logische Zeitstempel zur Einordnung/Reihenfolge

▷ Kausalitätsfunktion: $a \prec b$ (a passiert vor b) wenn:

(i) a, b im selben Prozess und a logisch im Code vor b

(ii) a ist Senden, b zugehöriges empfangen

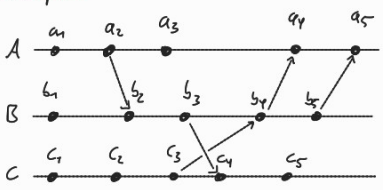
(iii) $\exists c: a<c \wedge c<b$

↳ Nebenläufig: $a \not< b \wedge b \not< a$

▷ Skalare Uhr: Jeder Prozess hat Zähler - liefert minimale Zeitdistanz an $TS(a) < TS(b)$ folgt nicht $a<b$

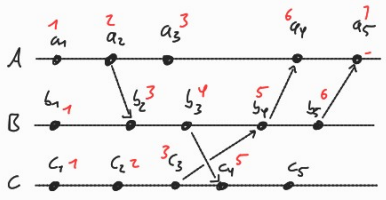
▷ Vektorzeitstempel: Jeder Prozess hat Vektor Länge n; Aus $VT(a) < VT(b)$ folgt $a<b$

Beispiel



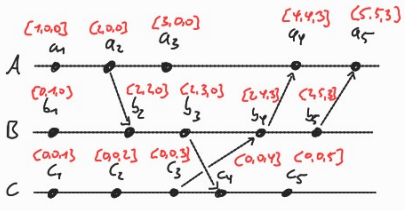
Skalarer Zeitstempel:

Prozess	Typ	TS	TS nach Berechnung → sendet
a_1	Lokal	0	0+1 = 1
a_2	sende → b_2	1	1+1 = 2
a_3	Lokal	2	2+1 = 3
b_1	Lokal	0	0+1 = 1
b_2	empfange ← a_2 (2)	1	max(1,2)+1 = 3
b_3	sende → c_4	3	3+1 = 4
c_1	Lokal	0	0+1 = 1
c_2	Lokal	1	1+1 = 2
c_3	sende → b_4	2	2+1 = 3
c_4	empfange ← b_3 (4)	3	max(3,4)+1 = 5
c_5	Lokal	5	5+1 = 6
b_4	empfange ← c_3(3), sende → a_4	4	max(4,3)+1 = 5
b_5	sende → a_5	5	5+1 = 6
a_4	empfange ← b_4(5)	3	max(3,5)+1 = 6
a_5	empfange ← b_5(6)	6	max(6,6)+1 = 7



Vektor Zeitstempel ($[V_a, V_b, V_c]$)

Prozess	Typ	TS	TS nach Berechnung → sendet
a_1	Lokal	[0,0,0]	V[0]++ = [1,0,0]
a_2	sende → b_2	[1,0,0]	V[0]++ = [2,0,0]
a_3	Lokal	[2,0,0]	V[0]++ = [3,0,0]
b_1	Lokal	[0,0,0]	V[1]++ = [0,1,0]
b_2	empfange ← a_2 ([2,0,0])	[0,1,0]	max([0,1,0],[2,0,0]) = [2,1,0]; V[1]++ = [2,2,0]
b_3	sende → c_4	[2,2,0]	V[1]++ = [2,3,0]
c_1	Lokal	[0,0,0]	V[2]++ = [0,0,1]
c_2	Lokal	[0,0,1]	V[2]++ = [0,0,2]
c_3	sende → b_4	[0,0,2]	V[2]++ = [0,0,3]
c_4	empfange ← b_3 ([2,3,0])	[0,0,3]	max([0,0,3],[2,3,0]) = [2,3,3]; V[2]++ = [2,3,4]
c_5	Lokal	[2,3,4]	V[2]++ = [2,3,5]
b_4	empfange ← c_3([0,0,3]), sende → a_4	[2,3,0]	max([2,3,0],[0,0,3]) = [2,3,3]; V[1]++ = [2,4,3]
b_5	sende → a_5	[2,4,3]	V[1]++ = [2,5,3]
a_4	empfange ← b_4([2,4,3])	[3,0,0]	max([3,0,0],[2,4,3]) = [3,4,3]; V[0]++ = [4,4,3]
a_5	empfange ← b_5([2,5,3])	[4,4,3]	max([4,4,3],[2,5,3]) = [4,5,3]; V[0]++ = [5,5,3]



Vergleich b_3, c_4:

$TS(b_3) = 4$, $TS(c_4) = 5$ → $4<5$ nicht zwingend $b_3 < c_4$: TS zwingt Totalordnung auf nebenläufige Ereignisse; Skalar vergibt jedem Ereignis eindeutige Zahl und erzeugt so Ordnung aller Ereignisse

↳ nebenläufige Ereignisse erhalten künstliche vor/nach Beziehung

$VT(b_3) = [2,3,0]$, $VT(c_4) = [2,3,4]$ → $2 \leq 2$, $3 \leq 3$, $0 < 4$ → $b_3 < c_4$

Stochastische Graphenmodelle

Nicht - Seriell - Parallel

▷ Gedächtnislosigkeit

Für $X \sim Exp(\lambda)$ gilt: $P(X > s+t \mid X > s) = P(X > t)$

„Egal wie lange ein Prozess bereits läuft – Restzeit immer wieder Exp(λ)-verteilt, als hätte er gerade erst angefangen"

↳ Reduktion auf eine Menge fertiger und laufender Knoten

▷ Zustandsgraphen erstellen: Zustand $z = (T(z) \mid F(z))$

↳ $F(z)$ Menge der bereits fertigen Teilaufgaben (finished)

↳ $T(z)$ Menge der aktiven Teilaufgaben (tasks) – Alle Knoten, deren sämtliche Vorgänger in S liegen, die aber selbst nicht in S liegen

↳ Startzustand: Alle Knoten ohne Vorgänger sind aktiv $F(z_0) = \emptyset$

↳ Endzustand: $T = \emptyset$, $F = V$ (alle fertig)

$k = |T(S)|$ – Anzahl gleichzeitig laufender Knoten
↳ Nebenläufigkeitsgrad in S

$Exp(k\lambda)$ – Verweildauer in S, bevor irgendein aktiver Knoten fertig wird

$p = 1/k$ – Wahrscheinlichkeit, dass genau einer der k aktiven Knoten als nächstes fertig wird

⇒ Knoten aktiv, wenn alle seine Vorgänger fertig

▷ Konstruktion:

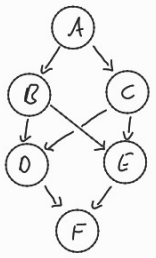
1) Startzustand $S_0 = \emptyset$ (noch nichts fertig)
↳ Bestimme $R(S_0)$ = Alle Knoten ohne Vorgänger

2) Für jeden aktiven Knoten in R(S):
Zeichne Kante zu einem neuen Zustand $S = S \cup \{\text{dieser Knoten}\}$ mit Wahrscheinlichkeit $1/|R(S)|$

3) Wiederhole für jeden neu entstandenen Zustand bis alle Knoten fertig erreicht

4) Zu jedem S: k_S (Anzahl aktiver Knoten), $\frac{1}{k_S \cdot \lambda}$ (Mittlere Verweildauer / Erwartungswert)

Beispiel



Gleichmäßige Exponentialverteilung

Nicht-Seriell-Parallel: B, C haben selben Vorgänger, aber nicht selbe Nachfolger

$S_0 = \emptyset$, $(S_0) = \{A\}$, $k_{S_0} = 1$, $Exp(\lambda)$, $p = 1$, $EW = 1/\lambda$, wkt: 1

$S_1 = \{A\}$, $T(S_1) = \{B, C\}$, $k_{S_1} = 2$, $Exp(2\lambda)$, $1/(2\lambda)$, 1

↳ $S_2 = \{A, B\}$, $T(S_2) = \{C\}$, $k_{S_2} = 1$, $1/\lambda$, $1/2$

↳ $S_{2'} = \{A, C\}$, $T(S_{2'}) = \{B\}$, $k_{S_{2'}} = 1$, $1/\lambda$, $1/2$

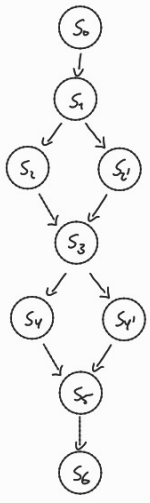
$S_3 = \{A, B, C\}$, $T(S_3) = \{D, E\}$, $k_{S_3} = 2$, $1/(2\lambda)$, 1

$S_4 = \{A, B, C, D\}$, $T(S_4) = \{E\}$, $k_{S_4} = 1$, $1/\lambda$, $1/2$

$S_{4'} = \{A, B, C, E\}$, $T(S_{4'}) = \{D\}$, $k_{S_{4'}} = 1$, $1/\lambda$, $1/2$

$S_5 = \{A, B, C, D, E\}$, $T(S_5) = \{F\}$, $k_{S_5} = 1$, $1/\lambda$, 1

S_6 = fertig



Maximale Nebenläufigkeit = $\max_S k_S = 2$

Gesamtbearbeitungszeit: $E[G] = \sum_{S \neq S_6} P(S) \cdot \frac{1}{k_S \cdot \lambda}$

$$E[G] = 1 \cdot \frac{1}{\lambda} + 1 \cdot \frac{1}{2\lambda} + \frac{1}{2} \cdot \frac{1}{\lambda} + \frac{1}{2} \cdot \frac{1}{\lambda} + 1 \cdot \frac{1}{2\lambda} + \frac{1}{2} \cdot \frac{1}{\lambda} + \frac{1}{2} \cdot \frac{1}{\lambda} + 1 \cdot \frac{1}{\lambda}$$

$$= \frac{2}{2\lambda} \cdot 2 + \frac{1}{2\lambda} \cdot 6 = \frac{10}{2\lambda} = \frac{5}{\lambda}$$

Serienparallele Reduktion

▷ Seriell verknüpft: Teilaufgaben v_i, v_j seriell verknüpft, wenn einzige Ausgangskante von v_i gleichzeitig einzige Eingangskante von v_j ist

▷ Parallel verknüpft: Teilaufgaben v_i, v_j parallel verknüpft, wenn sie dieselben Vorgänger und dieselben Nachfolger haben

▷ Seriell reduzierbar (serienparallel), wenn durch sukzessive Ersetzung auf einen einzigen Knoten reduziert werden kann

CONV → Serielle (sequentielle) Ausführung
⇒ Wenn B erst nach A starten kann → Zeiten addieren sich

Diskrete Verteilung: $P(A+B = k) = \sum_j P(A = j) \cdot P(B = k - j)$

„Für jede Möglichkeit wie A und B zusammen k ergeben; Einzelwahrscheinlichkeiten multiplizieren und aufaddieren"

$A: P(A=1) = \frac{1}{2}$, $P(A=2) = \frac{1}{2}$; $B: P(B=1) = \frac{1}{2}$, $P(B=2) = \frac{1}{2}$

k	Kombinationen	$P(A+B=k)$
2	$A=1, B=1$	$1/2 \cdot 1/2 = 1/4$
3	$A=1, B=2$; $A=1, B=1$	$1/2 \cdot 1/2 + 1/2 \cdot 1/2 = 1/2$
4	$A=2, B=2$	$1/2 \cdot 1/2 = 1/4$

MAX → parallele Ausführung

Wenn A, B gleichzeitig laufen und beide fertig sein müssen ⇒ max(A, B)

Formeln über kumulative Verteilungsfunktion

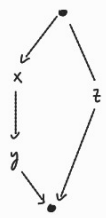
$P(\max(A,B) \leq k) = P(A \leq k) \cdot P(B \leq k)$ [Unabhängigkeit]

$P(\max(A,B) = k) = P(\max \leq k) - P(\max \leq k-1)$

$A: P(A=1) = 1/2$, $P(A=2) = 1/2$; $B: P(B=1) = 1$, $P(B=2) = 0$

k	$P(A \leq k) \cdot P(B \leq k)$	$P(\max = k)$
1	$1/2 \cdot 1 = 1/2$	$1/2 - 0 = 1/2$
2	$1 \cdot 1 = 1$	$1 - 1/2 = 1/2$

Beispiel



	1	2	3
X		1/3	2/3
Y	1/2	1/2	
Z	1/4		3/4

Tabelleneinträge: Einträge von Wahrscheinlichkeitsverteilungen für Bearbeitungszeiten
↳ Gesucht: Gesamtlaufzeit G

Graph ist serien-parallel reduzierbar: X → Y seriell, Z parallel
⇒ G = max(X + Y, Z)

Maximale Laufzeit = max(max(X) + max(Y), max(Z)) = max(X) + max(Y) = 3 + 2 = 5

1) conv(X,Y) = S:

$$P(S=s) = \sum_{x+y=s} P(X=x) \cdot P(Y=y),$$

$X \in \{2,3\}$, $Y \in \{1,2\}$

→ Mögliche Laufzeiten: 3, 4, 5:

k	paare (x,y)	wkt P(S=k)
3	(2,1)	$\frac{1}{3} \cdot \frac{1}{2} = \frac{1}{6}$
4	(2,2), (3,1)	$\frac{1}{3} \cdot \frac{1}{2} + \frac{2}{3} \cdot \frac{1}{2} = \frac{1}{2}$
5	(3,2)	$\frac{2}{3} \cdot \frac{1}{2} = \frac{1}{3}$

2) max(S, Z):

$P(\max(A,B) \le k) = P(A \le k) \cdot P(B \le k)$ [Unabhängigkeit]

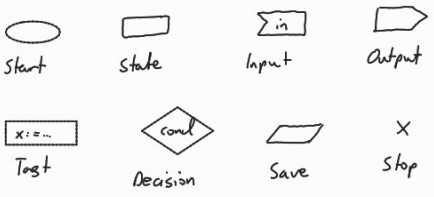
$P(\max(A,B) = k) = P(\max \le k) - P(\max \le k-1)$

k	P(S ≤ k)	P(Z ≤ k)	P(G ≤ k) = P(S ≤ k) · P(Z ≤ k)	P(G = k) = P(G ≤ k) − P(G ≤ k−1)
1	0	1/4	0	0
2	0	1/4	0	0
3	1/6	1	1/6	1/6 − 0 = 1/6
4	2/3	1	2/3	2/3 − 1/6 = 1/2
5	1	1	1	1 − 2/3 = 1/3

SDL / MSC

SDL-Symbolübersicht

SDL = Specification and description Language



Transition: Zustandswechsel ausgelöst durch konsumieren eines Input-Signals

SDL - Hierarchie

System: Oberste Ebene. Gesamtsystem mit Namen. Enthält Blöcke und Kanäle

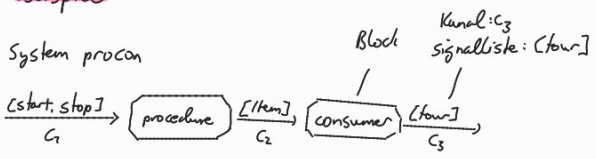
Block: Funktionaler Teilbereich des Systems. Enthält Prozesse und Prozeduren. Blöcke kommunizieren via Kanäle

Prozess: Aktive Komponente mit eigenem Zustandsautomaten
↳ Hat eigene Eingangswarteschlange. Kommuniziert via Signale

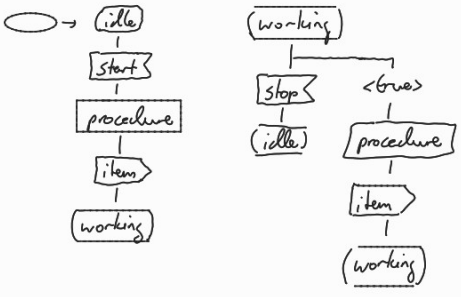
Prozedur: Teilverhalten eines Prozesses, das wiederverwendet werden kann; wie Funktion eingesetzt in SDL-Prozess

Beispiel

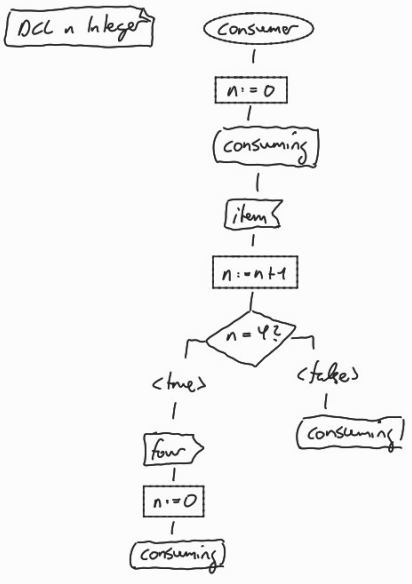
System procon



procedure:



Analoge consumer-Struktur:



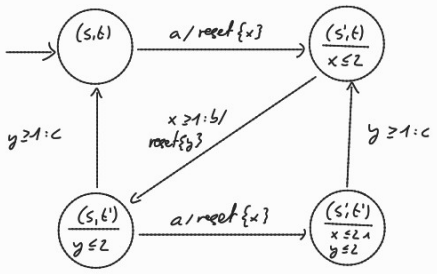
Timed Automat



Komposition $H_1 \parallel_{\{b\}} H_2$

↳ $\Sigma = \{b\}$ (synchronisiert), Produktzustände = $\{s,s'\} \times \{\epsilon,\epsilon'\} = 4$

Uhren = $\{x,y\}$, Invarianten = $I(p) \wedge I(q)$



▷ Komponenten:

Zustände - H_1: s, s' ; H_2: ϵ, ϵ'

Ankunftszustand - H_1: s ; H_2: ϵ

Uhren - H_1: $\{x\}$; H_2: $\{y\}$

Wächter (Guard) - Uhrbedingung auf Übergang
↳ Nur feuern wenn erfüllt

Reset-Menge - Uhren werden nach Übergang auf 0 gesetzt

▷ Übergangsnotation: [Guard :] Aktion [/ reset {Uhren}]

▷ Knoten-Invariante: Wartezeit $t \geq 0$ darf vergehen, solange Knoten-Invariante erfüllt (alle Uhren ≤ Schranke)

△ Komposition $\parallel_\Sigma$:

▷ Synchron ($a \in \Sigma$) - Beide haben a-Transition aus aktuellem Zustand
↳ $g_1 \wedge g_2$, reset $R_1 \cup R_2$, $(I_1, I_2) \to (I_1', I_2')$

▷ Asynchron ($a \notin \Sigma$) - Nur Automat H_i hat a-Transition
↳ $(I_1, I_2) \to (I_1', I_2)$

▷ Blockierend ($a \in \Sigma$) - Nur ein Automat hat a-Transition aus aktuellem Zustand
↳ Übergang nicht möglich